

Unidad 1 – Introducción a FastAPI

1 ¿Qué es FastAPI?

FastAPI es un **framework web moderno y de alto rendimiento para Python**, diseñado para crear APIs de forma rápida, segura y con documentación automática.

Está basado en dos pilares fundamentales:

- Starlette → manejo asíncrono y arquitectura ASGI.
- Pydantic → validación y modelado de datos.

2 ¿Por qué usar FastAPI?

FastAPI se ha convertido en uno de los frameworks más utilizados para backend moderno porque ofrece:

- ✓ Alto rendimiento (basado en ASGI)
- ✓ Validación automática de datos
- ✓ Documentación automática (Swagger y ReDoc)
- ✓ Uso intensivo de type hints (tipado estático)
- ✓ Código claro, mantenible y escalable

Arquitectura interna de FastAPI

Starlette: el motor asíncrono

Starlette es un toolkit ASGI que permite:

- Manejo de solicitudes HTTP
- Soporte para WebSockets
- Tareas en segundo plano
- Eventos de inicio y cierre
- Alta performance

FastAPI se construye sobre Starlette, por eso hereda su velocidad y arquitectura asíncrona.

Pydantic: validación de datos

Pydantic permite:

- Definir modelos usando clases Python
- Validar automáticamente datos de entrada

- Convertir datos a tipos correctos
- Generar esquemas para documentación

Ejemplo conceptual:

```
from pydantic import BaseModel
```

```
class Usuario(BaseModel):
```

```
    nombre: str
```

```
    edad: int
```

Esto garantiza que los datos recibidos cumplan el contrato definido.

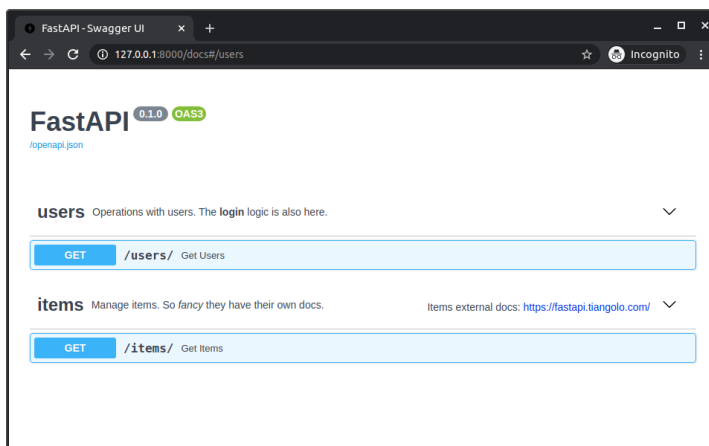
Documentación Automática

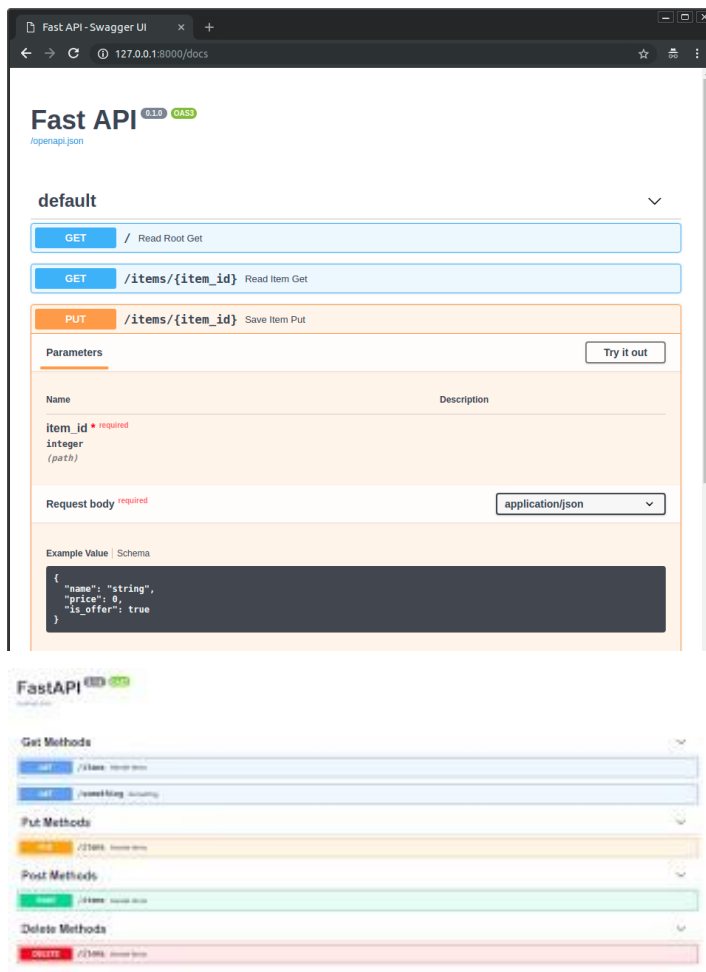
Una de las grandes ventajas de FastAPI es que genera documentación automática siguiendo el estándar OpenAPI.

Al ejecutar el servidor, se generan automáticamente:

- Swagger UI → /docs
- ReDoc → /redoc
- Esquema OpenAPI → /openapi.json

Swagger UI





Permite:

- Probar endpoints
- Ver parámetros
- Ejecutar solicitudes
- Generar código CURL automáticamente

Requisitos del Entorno

Para trabajar con FastAPI necesitamos:

- Python 3.10 o superior

- Visual Studio Code
- Entorno virtual
- Gestor de paquetes pip

Verificar versión de Python

En Linux/Mac: python3

--version

En Windows: py

--version

Entornos Virtuales en Python

Un entorno virtual es un **espacio aislado** donde se instalan dependencias sin afectar otros proyectos.

Permite:

- Tener diferentes versiones de librerías
- Evitar conflictos
- Mantener proyectos independientes

Creación del entorno virtual

Linux/Mac:

```
python3 -m venv .venv
```

Windows:

```
py -m venv .venv
```

Activación

Linux/Mac:

```
source .venv/bin/activate
```

Windows: .venv\Scripts\activate

Para desactivar: deactivate

Archivo requirements.txt

Es el archivo donde se declaran las dependencias del proyecto.

Ejemplo:

fastapi uvicorn

Instalación:

```
pip install -r requirements.txt
```

Al instalar FastAPI también se instalan:

- Uvicorn (servidor ASGI)
- Starlette • Pydantic

Primer Ejemplo: Hola Mundo

Archivo: main.py

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/") async def root():  
    return {"mensaje": "Hola Mundo"}
```

Explicación del código

Importación

```
from fastapi import FastAPI
```

Importa la clase principal del framework.

Instanciación app =

```
FastAPI()
```

Crea la aplicación.

Decorador de ruta

```
@app.get("/")
```

Indica:

- Método HTTP: GET

- Ruta: "/"

Función asíncrona

`async def root():`

Permite trabajar en modo asíncrono.

Respuesta `return {"mensaje":`

`"Hola Mundo"}`

- Devuelve un diccionario
- FastAPI lo serializa automáticamente a JSON

Ejecutar el servidor uvicorn

`main:app --reload` Explicación:

- `main` → nombre del archivo
- `app` → instancia de FastAPI
- `--reload` → recarga automática en desarrollo

Servidor disponible en:

`http://127.0.0.1:8000` Documentación:

`http://127.0.0.1:8000/docs`

Segundo Endpoint: /ping

`@app.get("/ping") async`

`def ping():`

`return {"status": "ok"}`

Este endpoint:

- Verifica que la API esté activa
- Devuelve estado simple

- Es común en arquitecturas backend

Pruebas de la API

Se puede probar:

- Desde Swagger (/docs)
- Desde navegador
- Con herramientas como Postman
- Con extensiones como REST Client
- Con CURL

Ejemplo:

```
curl http://127.0.0.1:8000/ping
```

Respuesta:

```
{  
  "status": "ok"  
}
```

Conceptos Clave Aprendidos

- ✓ FastAPI es moderno y rápido
- ✓ Usa Starlette y Pydantic
- ✓ Genera documentación automática
- ✓ Serializa automáticamente respuestas JSON
- ✓ Permite desarrollo asíncrono
- ✓ Se trabaja con entornos virtuales
- ✓ Uvicorn ejecuta el servidor